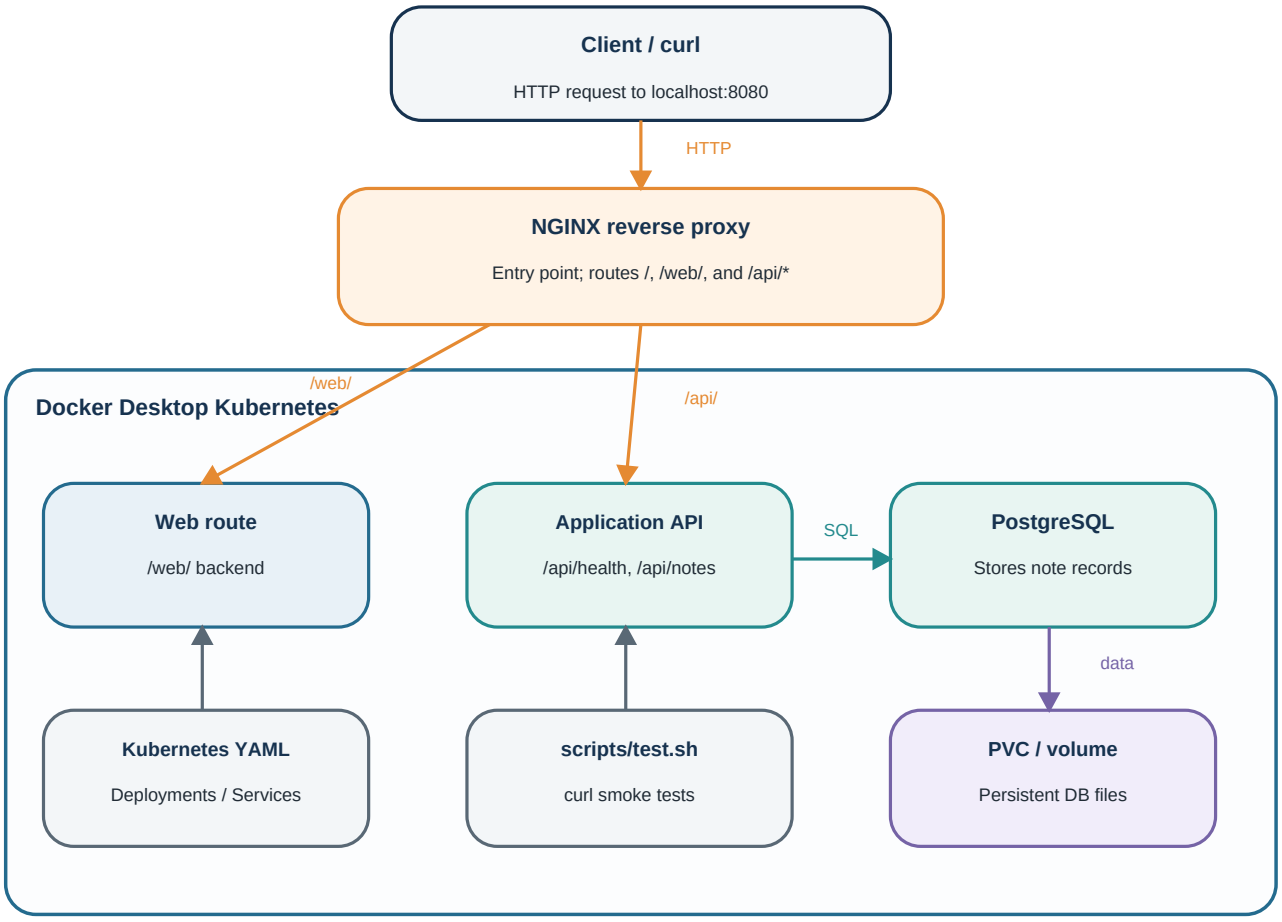
Kubernetes DevOps Assignment

Project architecture and implementation workflow



Architecture components

Component	Purpose	Why this tool/component is used
Docker Desktop Kubernetes	Runs Kubernetes locally on the Mac.	Avoids needing an AWS EC2 instance for this local assignment.
NGINX	Receives HTTP traffic and routes requests to the correct backend.	Provides one entry point and path-based routing.
Application / API	Serves the web route and API endpoints.	Implements the application behavior and database access.
PostgreSQL	Stores notes or other application records.	Provides persistent, structured data storage.
PersistentVolumeClaim	Requests storage for PostgreSQL data files.	Allows data to survive pod replacement when mounted/configured correctly.
Kubernetes YAML	Defines workloads, Services and storage resources.	Makes deployment declarative and repeatable.
scripts/test.sh	Calls HTTP routes with curl.	Quickly checks that the entry point, API health and web route respond.
Git	Tracks project files and commit history locally.	Records changes and supports review/version control.

Steps used to build and validate the project

1. Create the project structure

Created the project folder `kubernetes-devops-assignment` and organized files into `app/`, `k8s/`, `nginx/`, and `scripts/`. This keeps application code, Kubernetes manifests, proxy configuration, and tests separate.

2. Set up the local Kubernetes environment

Installed/used Docker Desktop and enabled its Kubernetes cluster. Used `kubectl get nodes` to check that a cluster node is available and reaches `Ready`. Docker Desktop keeps the assignment local; EC2 is not required unless cloud deployment is part of the requirements.

3. Prepare application and database resources

Used application files under `app/`, Kubernetes YAML under `k8s/`, and PostgreSQL with persistent storage. The PVC is intended to keep database files independent of an individual pod's lifecycle.

4. Configure NGINX routing

Configured NGINX files under `nginx/` to expose the application through a local HTTP entry point. The test workflow expects `http://localhost:8080` and checks `/`, `/api/health`, and `/web/`; `/api/notes` is the notes API route used to create/read records.

5. Deploy Kubernetes resources

Applied the manifests in `k8s/` using `kubectl apply -f ...` commands appropriate to the files. Checked pods, Services and PVC status with `kubectl get pods,services,pvc`. Services provide stable network endpoints for matching pods.

6. Validate HTTP endpoints

Created `scripts/test.sh` to run curl smoke tests against the root route, API health endpoint, and web route. `set -e` stops the script on the first failed command, helping identify which check needs troubleshooting.

7. Test database behavior and persistence

Used the notes API to create a test record and read it back. To validate persistence, restart/recreate the database pod in a controlled test and confirm the record remains; do not delete the PVC during this check.

8. Save work with local Git

Initialized a Git repository in the project folder, staged the project files with `git add .`, checked `git status`, and committed with a meaningful message. The local repository is stored in the project's hidden `.git/` directory; this does not upload code to GitHub.

Commands used during validation

```
kubectl get nodes
kubectl get pods -o wide
kubectl get deployments,services,pvc
kubectl logs <pod-name>
curl -i http://localhost:8080/api/health
curl -i http://localhost:8080/web/
./scripts/test.sh
git status
git log -1 --oneline
```

Why this toolset

Tool	Reason for using it
Docker Desktop + Kubernetes	Runs containerized workloads and a Kubernetes cluster locally for development/testing.
kubectl	Inspects cluster state and applies Kubernetes manifests.
NGINX	Routes HTTP traffic to the correct application route/backend.
PostgreSQL + PVC	Stores records and keeps database files on persistent storage.
curl + shell script	Automates repeatable HTTP smoke checks without needing a separate testing framework.
Git	Tracks changes and records local commits for version control.